WNDRVR
SUPPORT NETWORK

VXWORKS 7 THIRD-PARTY SOFTWARE SUPPORT, SR0650

Copyright Notice

Copyright © 2022 Wind River Systems, Inc.

All rights reserved. No part of this publication may be reproduced or transmitted in any form or by any means without the prior written permission of Wind River Systems, Inc.

Wind River, Simics, Tornado, and VxWorks are registered trademarks of Wind River Systems, Inc. Helix, Pulsar, Rocket, Titanium Cloud, Titanium Control, Titanium Core, Titanium Edge, Titanium Edge SX, Titanium Server, and the Wind River logo are trademarks of Wind River Systems, Inc. Any third-party trademarks referenced are the property of their respective owners. For further information regarding Wind River trademarks, please see:

www.windriver.com/company/terms/trademark.html

This product may include software licensed to Wind River by third parties. Relevant notices (if any) are provided for your product on the Wind River download and installation portals:

https://delivers.windriver.com/

https://windshare.usa.windriver.com/

Wind River may refer to third-party documentation by listing publications or providing links to third-party websites for informational purposes. Wind River accepts no responsibility for the information provided in such third-party documentation.

Corporate Headquarters

Wind River
500 Wind River Way
Alameda, CA 94501-1153
U.S.A.
Toll free (U.S.A.): +1-800-545-WIND
Telephone: +1-510-748-4100
Facsimile: +1-510-749-2010

For additional contact information, see the Wind River website:

www.windriver.com

For information on how to contact Customer Support, see:

www.windriver.com/support

VxWorks 7 Third-Party Software Support, SR0650

26 August 2020

1. ABOUT THIRD-PARTY SOFTWARE SUPPORT

1. VxWorks Support for Third-Party Software

Wind River provides VxWorks® 7 support for various third-party software applications.

This support is limited to extensions to the VxWorks 7 operating system, and applications that allow you to use the third-party software with VxWorks. Wind River makes no claims with regard to the third-party software itself.

If you are configuring support for third-party software on a system without Internet connectivity, you must obtain the required sources before you configure your VSB project.

The default location for the third-party download cache is *installDir* **/download**. If you make your *installDir* directory read-only, you must set the **WIND_3PP_DOWNLOAD_PATH** environment variable to an alternate directory location to store any downloads.

2. Source File Access in Systems without Internet Connectivity

If you are configuring support for third-party software on a system without Internet connectivity, you must obtain the required sources before you configure your VSB project.

Place the sources in the following directory:

installDir **/download**

If you make this *installDir* directory read-only, you must set the **WIND_3PP_DOWNLOAD_PATH** environment variable to an alternate directory location to store any downloads. Place the third-party source files in the alternate directory.

The following URLs are valid for specific software versions. Check the source makefile in the relevant third-party layer to verify the URL for your software version.

Third-Party Software	File Location
Boost on page 16	https://dl.bintray.com/boostorg/release/1.70.0/source/boost_1_70_0.tar.gz
ICU	https://github.com/unicode-org/icu/releases/download/release-65-1/icu4c-65_1-src.tgz
BZip2 on page 24	http://downloads.sourceforge.net/project/bzip2/bzip2-bzip2-1.0.6.tar.gz
Civetweb	https://github.com/civetweb/civetweb/archive/b0c7f652ac46b86a91e8316d21b9ef2d45b6f776.tar.gz
iPerf3 on page 4	**https://downloads.es.net/pub/iperf/iperf-***version***.tar.gz**

Third-Party Software	File Location
	Note: In addition, the iPerf recipe layer is not included with this release of VxWorks; you must download the layer from Wind River GitHub. **https://github.com/Wind-River/vxworks7-layer-for-iperf** Place the source code in the following directory: *installDir* **/vxworks-7/pkgs_v2/net**
Mosquitto	https://github.com/eclipse/mosquitto/archive/bf959ef9b0ae0e4d74bf80158ffb0b7c69da533d.tar.gz
open62541 on page 7	https://github.com/open62541/open62541/archive/v1.0.1.tar.gz
OpenCV on page 13	https://github.com/opencv/opencv/archive/a871f9e4f7d83dc5851f965bdba5cd01bb7527fe.tar.gz
OpenSSL	OpenSSL source files are present in VxWorks 7 at installation; however, if you need to restore or update the source files, you can obtain them from GitHub at the following location. https://github.com/openssl/openssl/archive/OpenSSL_1_1_1g.tar.gz Note: If the file that you obtain is named **openssl-OpenSSL_1_1_1g.tar.gz**, you must rename it as **OpenSSL_1_1_1g.tar.gz**.
Python 3.8 on page 29	https://www.python.org/ftp/python/3.8.0/Python-3.8.0.tgz
SQLite on page 32	https://www.sqlite.org/2019/sqlite-autoconf-3300100.tar.gz
libffi for the ctypes library on page 34	ftp://sourceware.org/pub/libffi/libffi-3.2.1.tar.gz.
NumPy on page 33	https://github.com/numpy/numpy/releases/download/v1.18.1/numpy-1.18.1.tar.gz
TPM2	https://github.com/tpm2-software/tpm2-tss/archive/c0889fcc76e5d5c48e984dcb626647542c817395.tar.gz https://github.com/tpm2-software/tpm2-tools/archive/bd4fc0cfc945f7cfd29d508f31352071ee07517e.tar.gz Makefiles: • http://opengrok.wrs.com/source/xref/vx7-integration/helix/guests/vxworks-7/pkgs_v2/security/tpm2_tss/tpm2_tss/Makefile • http://opengrok.wrs.com/source/xref/vx7-integration/helix/guests/vxworks-7/pkgs_v2/security/tpm2_tss/tpm2_tools/Makefile
mimalloc	https://github.com/microsoft/mimalloc/archive/v1.1.0.tar.gz
OPC UA on page 7	https://github.com/OPCFoundation/UA-AnsiC-Legacy/archive/a3d9120423f9dac0aa784d2f15db440c083d0ae0.tar.gz

Third-Party Software	File Location
UTILS_ARCHIVE	https://github.com/libarchive/libarchive/archive/3288ebb0353beb51dfb09d444dedbe9235ead53d.tar.gz
TILS_TBB	https://github.com/01org/tbb/archive/e8b3e241ca2c006919797dadfa3bdc91b93bf690.tar.gz

2. IPERF3

1. iPerf3 and VxWorks

VxWorks provides support for iPerf3. iPerf3 is a tool for active measurements of the maximum achievable bandwidth on IP networks. It supports tuning of various parameters related to timing, buffers and protocols (TCP, UDP, SCTP with IPv4 and IPv6).

For general information about iPerf3, see http://software.es.net/iperf/.

For more information about the VxWorks recipe layer for iPerf3, see https://github.com/Wind-River/vxworks7-layer-for-iperf.

2. Configuring VxWorks with iPerf3 Support

VxWorks support for iPerf3 is provided with VSB and VIP options. This support also includes the iPerf3 command.

Before You Begin

- Ensure the Unix compatibility layer is supported in your VxWorks release. The layer provides support for porting third-party code to VxWorks.
- Ensure the iPerf3 source code is available from the following location:

  https://downloads.es.net/pub/iperf/iperf-*version* **.tar.gz**
- If you are on a system without Internet connectivity, you must obtain the iPerf3 source code from the above location and place it in the following directory:

  installDir **/download**

  If you make this *installDir* directory read-only, you must set the **WIND_3PP_DOWNLOAD_PATH** environment variable to an alternate directory location to store any downloads. Place the third-party source files in the alternate directory.

  You must also download the the iPerf recipe layer from Wind River GitHub at the following location:

  https://github.com/Wind-River/vxworks7-layer-for-iperf

  Place the recipe layer in the following directory:

  installDir **/vxworks-7/pkgs_v2/net**
- Ensure that **patch** is installed in addition to the regular build support for VxWorks.

Procedure

1. If you are on a system with Internet connectivity, use the checkout feature of GitHub to place the contents of the following repository in your VxWorks install tree at an appropriate location.

   Check out:

   https://github.com/Wind-River/vxworks7-layer-for-iperf

   To, for example: *installDir* **/vxworks-7/pkgs_v2/net**

2. Add the **IPERF3** layer, using vxprj or other appropriate tool.

   For example:

```
$ cd VSB_DIR
$ vxprj vsb add IPERF3
$ make
```

3. Build the VSB project.

   During the build, the iPerf3 source is downloaded (or accessed from the *installDir* **/download** directory) and placed in the VSB directory:

   vsbDir **/3pp/IPERF3/***iperf-version*

   The source is patched for VxWorks and the RTP and libraries are built and placed in the standard locations in the VSB directory tree.

4. Create a VIP project.
5. Add the **INCLUDE_IPERF3** component.

   For example:

```
$ cd VIP_DIR
$ vxprj component add INCLUDE_IPERF3
$ make
```

   This component makes the iPerf3 shell command available from the VxWorks C interpreter.

6. Optional: If you would like to access iPerf3 through the VxWorks command-line shell, add the **INCLUDE_IPERF3_CMD** component.

   This component makes the iPerf3 shell command available from the VxWorks command-line shell.

7. Build the VIP project.

3. Running iPerf3

You can run iPerf3 using either the VxWorks C interpreter or command-line shell.

About This Task

To run iPerf3 from the VxWorks C interpreter, you must first include the **INCLUDE_IPERF3** component in your VIP project.

To run iPerf3 from the VxWorks command-line, you must include both **INCLUDE_IPERF3** and **INCLUDE_IPERF3_CMD**.

It is not necessary to perform both of the following steps. You can run iPerf3 by performing either one.

Procedure

1. To run the iperf3 server in the background, use the VxWorks sp (spawn) command:

```
-> sp iperf3,"-s -B 0.0.0.0"
```

   This command spawns an iPerf3 server in the background on all local IPv4 addresses.

WNDRVR

2. You can also run iPerf3 as a client on the VxWorks command-line shell:

```
[vxWorks *]# iperf3 -c192.168.200.254 -t3
Connecting to host 192.168.200.254, port 5201
[  8] local 192.168.200.10 port 58786 connected to 192.168.200.254
port 5201
[ ID] Interval           Transfer     Bandwidth
[  8]   0.00-1.00   sec  20.1 MBytes   169 Mbits/sec
[  8]   1.00-2.00   sec  20.2 MBytes   169 Mbits/sec
[  8]   2.00-3.00   sec  20.1 MBytes   169 Mbits/sec
- - - - - - - - - - - - - - - - - - - - - - - - -
[ ID] Interval           Transfer     Bandwidth
[  8]   0.00-3.00   sec  60.4 MBytes   169 Mbits/sec
sender
[  8]   0.00-3.00   sec  60.4 MBytes   169 Mbits/sec
receiver

iperf Done.
[vxWorks *]#
```

3. OPEN62541

1. open62541 Support

VxWorks provides support for OPC Unified Architecture (UA) in the form of open62541, an open source and free implementation of the OPC UA protocol stack.

The OPC UA protocol stack is a platform-independent, service-oriented architecture that supports open connectivity and interoperability between devices in industrial domains. The previously supported OPC UA protocol stack provided by the OPC foundation is in legacy status; it is no longer supported in VxWorks.

With open62541, you can integrate the OPC UA protocol stack into your applications to support communication between devices. The open62541 library is compatible with all major compilers and is written in the common subset of the C99 and C++98 languages. For general information about open62541, see https://open62541.org/.

open62541 is a third-party software library licensed under the Mozilla Public License v2.0. If you include this library in your project, the library will be downloaded and installed from GitHub.

VxWorks supports the use of an OPC UA protocol stack in user space using real-time process (RTP) applications or in kernel space. It also provides sample client, server, publisher, and subscriber applications for you to test your applications.

2. Configuring VxWorks with open62541 Support

To configure VxWorks with support for open62541, you must select the required layer and components for your VxWorks source build (VSB) projects and VxWorks image projects (VIPs).

Before You Begin

- You must know how to use Workbench or the vxprj utility to create and configure VSB and VIP projects. For more information, see the VxWorks 7 Configuration and Build Guide.
- If you are on a system without Internet connectivity, you must obtain the open62541 sources and put them in the following directory:

  installDir **/download**

  If you make this *installDir* directory read-only, you must set the **WIND_3PP_DOWNLOAD_PATH** environment variable to an alternate directory location to store any downloads. Place the third-party source files in the alternate directory.

  The open62541 version 1.0.1 sources are available from the following location:

  https://github.com/open62541/open62541/archive/v1.0.1.tar.gz

  Check the source makefile in the **OPEN62541** layer to verify the URL for your version of open62541.

Procedure

1. Create a VSB project based on your board support package (BSP).
2. Enable the **OPEN62541** layer.

The open62541 source is downloaded (or accessed from the *installDir* **/download** directory) and placed in the following directory:

vsbDir **/3pp/OPEN62541/open62541-***version*

The source is patched for VxWorks, and the RTP and libraries are built and placed in the VSB directory.

3. Enable any additional options that your application requires.

   For example, if you want to use subscriber and publisher servers, you must include the **OPEN62541_UA_ENABLE_PUBSUB** option. For information on additional open62541 configuration options, see open62541 Project Configuration Reference on page 8.

4. Build the VSB project.
5. Create a VIP based on the VSB project.
6. Build the VIP.

3. open62541 Project Configuration Reference

VxWorks provides several configuration options for VxWorks source build (VSB) projects to enable features in open62541 applications.

See the open62541 build options documentation at https://open62541.org/doc/current/toc.html for additional information about the following configuration options.

Option	Description	Default Status
OPEN62541_UA_ENABLE_DISCOVERY	Enable the local discovery server (LDS) to publically expose any open62541 servers	Enabled
OPEN62541_UA_ENABLE_HISTORIZING	Enable servers and clients to provide access to historical data	Disabled
OPEN62541_UA_LOGLEVEL	Set logging event levels **600**: Fatal **500**: Error **400**: Warning **300**: Info **200**: Debug **100**: Trace	**300**: Error
OPEN62541_UA_ENABLE_NODEMANAGEMENT	Enable the dynamic addition and removal of nodes at runtime	Enabled
OPEN62541_UA_ENABLE_PUBSUB	Enable the use of publishers and subscribers	Disabled
OPEN62541_UA_ENABLE_QUERY	Enable clients to issue queries to servers	Disabled
OPEN62541_UA_ENABLE_SUBSCRIPTIONS_EVENTS	Enable subscribers to respond to publisher events	Disabled

4. Adding OpenSSL Encryption Support to open62541

You can secure your OPC UA protocol stack applications by adding OpenSSL encryption support to your open62541 VxWorks project.

About This Task

You can also add and run encryption client and server examples.

Before You Begin

Configure your VSB and VIP projects to support **open62541** as described in Configuring VxWorks with open62541 Support on page 7.

Procedure

1. Under the **OPEN62541** component in your VSB, enable **OPEN62541_UA_ENABLE_ENCRYPTION_OPENSSL**
2. Rebuild the VSB.
3. (Optional) To include client and server examples for open62541 encryption, enable these components in your VIP:
   - **INCLUDE_OPEN62541_CLIENT_ENCRYPTION**
   - **INCLUDE_OPEN62541_SERVER_ENCRYPTION**
4. Rebuild the VIP.

   To run the client example in a target:

   ```
   -> client_encryption ("<opc.tcp://host:port> <client-certificate.der> <client-private-key.der>
   [<trustlist1.crl>, ...]")
   ```

   To run the server example:

   ```
   -> server_encryption ("<server-certificate.der> <private-key.der> [<trustlist1.crl>, ...]")
   ```

5. Creating open62541 RTP Applications

To create an open62541 application that runs in user space, you must create and configure a VxWorks real-time process (RTP) application.

About This Task

If you want to test your application with an existing client, server, publisher, or subscriber application, you can run any of the following open62541 sample RTP applications:

- **open62541Client**
- **open62541Server**
- **open62541Pub**
- **open62541Sub**

The sample applications are located in the following directory:

workspace/*vsbDir*/**usr/root/***cpuTool*/**bin**

Before You Begin

- You must have configured the VxWorks source build (VSB) project that your application will be based on. For more information, see Configuring VxWorks with open62541 Support on page 7.
- You must know how to create an RTP project. For more information, see the following tutorials:
  - Real-Time Process Application Tutorial Using Workbench 4
  - Real-Time Process Application Tutorial from the Command Line

Procedure

1. Create an RTP project based on the VSB that you created in Configuring VxWorks with open62541 Support on page 7.
2. Add the **-lopen62541** and **-lnet** libraries to the project:
   1. In Workbench, right-click the RTP project and select **Properties**.
   2. Select **Build Properties** and select the **Libraries** tab.
   3. Add the **-lopen62541** library file.
   4. Add the **-lnet** library file.
   5. Click **Apply**.
3. Add the **-DUA_ARCHITECTURE_VXWORKS** macro to the project:
   1. Select the **Defines** tab.
   2. Add the **-DUA_ARCHITECTURE_VXWORKS** macro.
   3. Click **Apply and Close**.
4. Add source code to your application.
5. Build the project.

Results

You can access either your new application or the sample applications by adding the relevant RTP executable to a ROMFS in your project; for more information, see the *VxWorks 7 Kernel Application and RTP Application Projects Guide:* ROMFS File System Projects.

6. Running Sample Client and Server Kernel Applications

VxWorks provides sample open62541 applications with which you can test client or server applications.

About This Task

You can add the open62541 client component or the open62541 server component to the VxWorks image project (VIP) to run the associated application in kernel space.

To run the sample applications in user space, see Creating open62541 RTP Applications on page 9.

Before You Begin

- You must know how to use Workbench or the vxprj utility to create and configure VxWorks source build (VSB) projects and VxWorks image projects (VIPs). For more information, see the VxWorks 7 Configuration and Build Guide

- You must know how to connect to your target. For more information, see VxWorks 7 Demo Series: Using Workbench to Connect to a VxWorks Target.

Procedure

1. Create a VSB project based on your board support package (BSP).
2. Enable the **OPEN62541** layer.
3. Build the VSB project.
4. Create a VIP based on the VSB project.
5. Add one of the following components to the VIP.

Option	Description
INCLUDE_OPEN62541_SERVER	Add this component if you want to run a sample server application that communicates with your client application.
INCLUDE_OPEN62541_CLIENT	Add this component if you want to run a sample client application that communicates with your server application.

6. Build the VIP.
7. Connect to your target.

   If you added the **INCLUDE_OPEN62541_SERVER** component to the VIP, the sample server application starts automatically when you connect to the target. You can use your client application to communicate with the server.

   If you added the **INCLUDE_OPEN62541_CLIENT** component to the VIP, perform the following step to run the client.

8. On the target terminal, run the sample client.

```
-> sp open62541Client, "targetIP"
Task spawned: id = 0x400e5e0d0, name = t1
vTahleu ee n=d  17194934480p o=i n0tx 400e5e0d0is: opc.tcp://127.0.0.1:4840
[2020-01-17 13:42:39.660 (UTC+0000)] info
...
```

   In the above command, *targetIP* is the IP address of the server; for the sample server application, this is **127.0.0.1**.

7. Running Sample Publisher and Subscriber Kernel Applications

VxWorks provides sample open62541 applications with which you can test a publisher or subscriber application.

About This Task

You can add the open62541 publisher component or the open62541 subscriber component to the VxWorks image project (VIP) to run the associated application in kernel space.

To run the sample applications in user space, see Creating open62541 RTP Applications on page 9.

WNDRVR

Before You Begin

- You must know how to use Workbench or the vxprj utility to create and configure VxWorks source build (VSB) projects and VIPs. For more information, see the VxWorks 7 Configuration and Build Guide
- You must know how to connect to your target. For more information, see VxWorks 7 Demo Series: Using Workbench to Connect to a VxWorks Target.

Procedure

1. Create a VSB project based on your board support package (BSP).
2. Enable the **OPEN62541** layer.
3. Build the VSB project.
4. Create a VIP based on the VSB project.
5. Add one of the following components to the VIP.

Option	Description
INCLUDE_OPEN62541_SUBSCRIBER	Add this component if you want to run a sample subscriber application that subscribes to your publisher application.
INCLUDE_OPEN62541_PUBLISHER	Add this component if you want to run a sample publisher application that your subscriber application can subscribe to.

6. Build the VIP.
7. Connect to your target.

   If you added the **INCLUDE_OPEN62541_SUBSCRIBER** component to the VIP, the sample subscriber application starts automatically when you connect to the target.

   If you added the **INCLUDE_OPEN62541_PUBLISHER** component to the VIP, perform the next step to start the publisher application.

8. On the target terminal, run the sample publisher application.

```
-> sp open62541PubSub_Pub, "uri", "niclP"
Task spawned: id = 0x400e5e0d0, name = t1
vTahleu ee n=d  17194934480p o=i n0tx 400e5e0d0is: opc.tcp://127.0.0.1:4840
[2020-01-17 13:42:39.660 (UTC+0000)] info
...
```

   In the above command, *uri* is the publisher and subscriber URI; for the sample applications, this is **opc.udp://224.0.5.1:4840**. *niclP* is the subscriber NIC IP address; for the sample subscriber application, this is **127.0.0.1**.

4. OPENCV

1. OpenCV Support

VxWorks provides support for Open Source Computer Vision (OpenCV) on ARM and Intel processors, with minor limitations.

OpenCV is an open-source computer vision and machine learning software library (see https://opencv.org).

VxWorks provides support for OpenCV in the form of user mode (RTP) libraries. For IA, a static library and a shared library are supported for 32-bit and 64-bit processors. For ARM, a static library and a shared library are supported for 32-bit, and a static library is supported for 64-bit.

The libraries can be built with the GNU or LLVM compilers. For LLVM, the build has been verified for ARM 64-bit.

VxWorks support for OpenCV is based on OpenCV 3.3.1. It has the following limitations:

- Not all modules are supported. For a list of all OpenCV modules, see https://docs.opencv.org/3.3.1/index.html. For those that are included in the VxWorks OpenCV library, see the **VPATH** assignment in **vxworks.lib.mak** (in *installDir* **/vxworks-7/pkgs_v2/app/opencv/usr_src/vxworks.lib.mak**).
- For writing video files and capturing from video files, only single channel MJPEG-encoded videos are supported (that is, without sound).
- For writing and reading image files, only JPEG images are supported.

Threading Building Blocks (TBB) should only be used with OpenCV. It is not supported for other purposes.

2. Configuring VxWorks with OpenCV Support

Create a VSB project with OpenCV support and other options required for your system. Create a VIP application based on your VSB project, and configure it as required for your system.

Before You Begin

If you are on a system without Internet connectivity, you must obtain the OpenCV sources and put them in the following directory:

installDir **/download**

If you make this *installDir* directory read-only, you must set the **WIND_3PP_DOWNLOAD_PATH** environment variable to an alternate directory location to store any downloads. Place the third-party source files in the alternate directory.

Obtain the OpenCV sources from the following location:

https://github.com/opencv/opencv/archive/a871f9e4f7d83dc5851f965bdba5cd01bb7527fe.tar.gz

Check the source makefile in the **OpenCV** layer to verify the URL for your version of OpenCV.

Procedure

1. Create a VSB project and add the OPENCV layer.
2. Add any other layers that are required for you system.

For example, for IA architectures, you may want to include:
- USB_UVC for USB video cameras.
- FBDEV_ITLGMCFB for frame buffer devices.

For ARM, you may want to include:
- USB_UVC for USB video cameras.
- USB_EHCI for the enhanced host controller interface.
- FBDEV_FSLIPUFB for frame buffer devices on the i.MX6 platform.

3. Build the VSB project.
4. Create a VIP project based on your VSB.
5. Add **INCLUDE_POSIX_PTHREAD_SCHEDULER**.
6. Add components for a storage facility.

   If you are going to use ROMFS, create a ROMFS project that is subordinate to your VIP project.

7. Add any other components that your system requires.
8. Build the VIP.

3. Creating an OpenCV RTP Application

Create an RTP application that is either statically linked with the OpenCV library, or a dynamic executable that uses OpenCV and other shared libraries.

About This Task

VxWorks example code and instructions are written (as part of the VSB build) to *vsbDir* **/3pp/OPENCV/opencv-3.3.1/vxworks_examples**.

Procedure

1. Create an RTP project based on your VSB.
2. Replace the default **rtp.c** with your own C or C++ files.
3. Select a C++2011 build spec for your compiler.
   1. From Workbench, select the RTP project name in Project Explorer, and from the context menu select **Properties > Build Properties**.
   2. Select the C++2011 build spec for your compiler from the **Active build spec** drop-down list.
   3. Click **Apply**.
4. If you want to build a statically-linked RTP, add the required linkage option.
   1. From the Build Properties dialog, select **Build Properties > Libraries**.
   2. Add -L *vsbDir* **/usr/lib/common**, -l opencv, -l tbb, and -l gfxJpeg.
   3. Click **Apply** and **OK**.
5. If you want to build your RTP application for use with shared libraries, configure the RTP build for a dynamic executable and add the required options for dynamic linkage.
   1. From the Build Properties dialog, select the **Tools** tab.
   2. From the Build tool drop-down list, select **Linker**.
   3. Select **Tools Flags**.
   4. In the Linker Options dialog, select the Create a dynamic executable checkbox, and click **OK**.
   5. In the Build Properties dialog, select the **Variables** tab.

   6. In the Build Spec Specific Settings panel, select the LIBS entry in the table.
   7. Click **Edit**, add these options in front of what is already there: -lopencv -ltbb -lgfxJpeg. Then click **OK**.
   8. Click **Apply** and **OK**.
6. If are using a monitor with your application, add the required support code.

   Add the *vsbDir* **/3pp/OPENCV/opencv-3.3.1/vxworks_examples/fb_output** directory to your RTP project. If you add it outside of Workbench, refresh your Workbench project after you have done so.

7. Build the RTP project.

   When you start the build, select **Generate Includes** when the Build dialog box appears to set the correct header file include paths.

Next Steps

- If the RTP application is a dynamic executable, add the shared libraries that are built in *vsbDir* **/usr/root/***compiler* **/bin** to the same storage directory as the RTP application (or use other means of identifying their location; see the *VxWorks 7 Application Programmer's Guide:* Shared Library Location and Loading at Run-time).
- If you are using ROMFS for storage, add your application to the ROMFS project (and the shared libraries, if it is a dynamic executable). Then rebuilt the VIP.

5. BOOST

1. Boost

VxWorks provides support for Boost C++ libraries.

The Boost project provides free peer-reviewed portable C++ source libraries. Boost emphasizes libraries that work well with the C++ Standard Library. Boost libraries are intended to be widely useful and usable across a broad spectrum of applications. The Boost license encourages both commercial and non-commercial use.

Boost consists of many libraries, not all of which work with VxWorks. Some require C++ support not available in VxWorks or are dependent on other libraries or functionality not available for VxWorks. Wind River's goal is to validate as many Boost libraries as possible and then push the required changes to boost.org. Changes not yet incorporated in the public version of Boost are provide in the VxWorks 7 Boost layer as patches, which are applied before the libraries are built and installed.

Check the *VxWorks 7 Release Notes: Boost C++ Libraries* for the Boost libraries that have been validated for the release you are working with. All Boost libraries are installed, but Wind River documents in the release notes the libraries that have successfully passed testing.

2. Configuration and Build

Build the Boost layer to download, unpack, build, and install Boost libraries.

Before You Begin

If you are on a system without Internet connectivity, you must obtain the Boost sources and put them in the following directory:

installDir **/download**

If you make this *installDir* directory read-only, you must set the **WIND_3PP_DOWNLOAD_PATH** environment variable to an alternate directory location to store any downloads. Place the third-party source files in the alternate directory.

On a Linux build host, obtain these sources from https://dl.bintray.com/boostorg/release/1.71.0/source/boost_1_71_0.tar.gz, and on Windows, from https://dl.bintray.com/boostorg/release/1.71.0/source/boost_1_71_0.tar.gz.

Check the source makefile in the **Boost** layer to verify the URL for your version of Boost.

If you are online, these are downloaded by wget during the build.

About This Task

Boost is primarily a set of C++ template header libraries. For the majority of libraries, the build stage just installs Boost headers into the VSB.

However, some boost libraries have a separately compiled source component. The compiled libraries are built and installed if added to the VSB configuration.

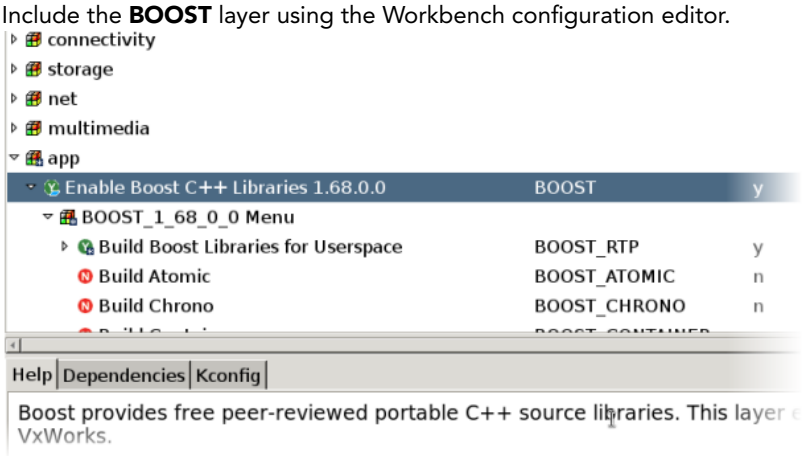
Procedure

1. Create a VxWorks source build (VSB) project based on your BSP with Workbench or from the command line.

For example, with the Linux simulator BSP:

```
$ cd installDir
$ ./wrenv.linux -p helix
$ cd ../workspace
$ export WIND_WRTOOL_WORKSPACE=`pwd`
$ wrtool prj vsb create -bsp vxsim_linux simVSB
$ cd simVSB
```

2. Include the **BOOST** layer using the Workbench configuration editor.



3. Include the Boost libraries you wish to build.

   You can also configure and build the VSB using the command line (in Workbench or in a separate wrenv shell) with vxprj or wrtool.

   For example, with vxprj:

```
$ vxprj vsb add BOOST
$ vxprj vsb config -s -add "_WRS_CONFIG_BOOST_MATH=y" \
     -add "_WRS_CONFIG_DEFAULT_THREAD_PRIO_INHERIT=y" \
     -add "_WRS_CONFIG_BOOST_THREAD=y"
$ make
```

   For example, with wrtool:

```
$ wrtool prj vsb add BOOST
$ wrtool prj vsb value set BOOST_MATH y
$ make
```

4. If you are using Workbench, turn off C++ indexing before you build the VSB, as the Boost libraries are large and will exceed the JVMs memory:
   1. Right-click the VSB project name and select **Properties**.
   2. Expand **C/C++ General** and select **Indexer**.
   3. Clear the **Enable Indexer** check box and click **Apply and Close**.
5. Build the project.

   Only the Boost **system** library is included by default. Some Boost libraries are dependent on other Boost libraries. You may wish to enable all of the *non-TEST* Boost VSB components because it is not always initially obvious which libraries are required.

Note: The **BOOST_THREAD** component has an additional dependency. You must enable **DEFAULT_PTHREAD_PRIO_INHERIT** before the Boost thread library is visible in the VSB configuration editor.

The **BOOST_IOSTREAMS** componet has an additional dependency. You must enable **BZIP2** before the Boost IOstreams library is visible in the VSB confguration editor.

Boost will be downloaded from the internet if necessary. The build will unpack Boost and apply patches to add VxWorks Support. The VSB build invokes the Boost supplied build system, wrapping it in the cross-build environment. The makefile that actually builds the code is **vxworks.lib.mak**. This is copied to the Boost build directory, *vsbDir* **/3pp/BOOST/boost_1_71_0/**. The Boost's build system **install** stage will copy the headers and libraries to the standard locations in the VSB.

VxWorks Image Projects on page 18 There is no specific component for Boost.

2.1. VxWorks Image Projects

There is no specific component for Boost.

Boost is many different libraries with different dependencies. Different libraries require different components. For example, the popular **asio** library requires a full network stack and socket support; however, most other libraries have no dependency on networking. Note that most libraries require POSIX functionality.

The following components are recommended:

- **INCLUDE_PX_SCHED_DEF_POLICIES**
- **INCLUDE_POSIX_PIPES**
- **INCLUDE_POSIX_SHM**
- **INCLUDE_CPLUS_DEMANGLER**
- **INCLUDE_DEV_URANDOM**

Parent topic: Configuration and Build on page 16

3. Incremental Builds

Boost can be built using the layer build command.

For example, from the command line:

```
$ vxprj vsb build BOOST
```

Note: Before doing an incrimental build, first build the complete VSB project with BOOST included.

No clean rule is provided for the Boost layer, but cleaning the VSB rebuilds Boost as well. The download directory is not touched; therefore, the rebuild begins from the unpack stage.

Removing the *vsbDir* **/3pp/BOOST** directory or *vsbDir* **/3pp/BOOST/boost_1_71_0/bin.v2/libs/***library_name* , and then building the VSB, incrementally rebuilds Boost, without rebuilding any other libraries.

4. Using the b2 utility directly

If you have built Boost for other operating systems, you will have used the b2 build utility directly; this is also possible with VxWorks.

The initial build of the Boost layer creates the **vxworks_env.sh** script, which contains the current VSB cross build environment. This is propagated to the Boost jam build environment by the generic **project_config.jam** file provided in the Boost layer. Once these files are in place, you can optionally invoke b2 outside of the VSB build by first sourcing **vxworks_env.sh**. Duplicate the command found in the build log. For example:

```
$ cd  3pp/BOOST/boost_1_71_0
$ . ./vxworks_env.sh
$ ./b2 install --prefix=VSBDir/usr/root --libdir=VSBDir/usr/lib/common \
     --includedir=VSBDir/usr/h/public  link=static  toolset=gcc cross-compile=vxworks \
     --with-atomic --with-chrono --with-context --with-coroutine --with-exception \
     --with-test -q -d2
```

5. Using Boost Libraries with RTP Applications

Boost libraries can be used in RTP applications.

About This Task

After building your VSB project with the Boost layer, you can use the boost libraries in an RTP project.

Procedure

1. Create the project and base it on the VSB project that you created in Configuration and Build on page 16.
2. Add your code to the project.

   For example, you could use the code provided at http://www.boost.org/doc/libs/1_63_0/libs/timer/doc/cpu_timers.html.

   > Note: Make sure that your file extension is **.cpp** rather than **.c**.

3. Add the required library directives in the RTP project build properties:
   1. If you are using Workbench, right-click the RTP project name and select **Properties**.
   2. Select **Build Properties** and select the **Libraries** tab.
   3. Add the **boost_system** and **unix** libraries for all applications, as well as any other libraries that are specifically required for your application to build.

      For the example code mentioned above, add:

      ```
      -lboost_system
      -lboost_timer
      -lboost_chrono
      -lunix
      ```

   4. Click **Apply and Close**.
4. The Boost headers are copied into the VSB's normal include paths (at **boost/...**); therefore, you should not need to add any include paths specifically for Boost. You may need to add an include path for the published headers in the **UNIX** layer.
   1. Using Workbench, right-click the RTP project name and select **Properties**.
   2. Select **Build Properties** and select the **Paths** tab.

3. Add the following directive:

```
-I$(VSB_DIR)/usr/h/published/UTILS_UNIX
```

4. Click **Apply and Close**.

5. Build the RTP project.

6. Adding ICU Support for Boost.Locale

Include the ICU layer in your VSB to include ICU libraries that are used with Boost.Locale.

About This Task

Boost.Locale uses the ICU library to enhance and unify APIs for the development of cross platform software. For more information, see the Boost.Locale site.

International Components for Unicode (ICU) is an open-source project of C/C++ and Java libraries for Unicode support for software internationalization and globalization. ICU gives applications the same results on all platforms including C, C++, and Java software. You can also use the ICU C interface to develop your VxWorks projects if you prefer it over the C++ interface. For more information, see the ICU site.

Before You Begin

- If you are on a system without Internet connectivity, you must obtain the ICU source code from the below location and place it in the following directory:

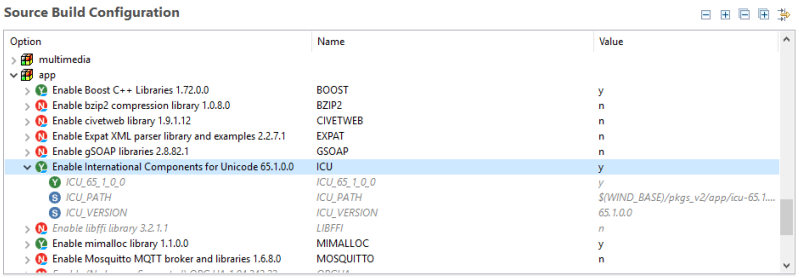
  installDir **/download**

  ICU source download location: https://github.com/unicode-org/icu/releases/download/release-65-1/icu4c-65_1-src.tgz

  If you make this *installDir* directory read-only, you must set the **WIND_3PP_DOWNLOAD_PATH** environment variable to an alternate directory location to store any downloads. Place the third-party source files in the alternate directory.
- Create a VxWorks source build (VSB) project based on your BSP with Workbench or from the command line. For more information, see the VxWorks 7 Configuration and Build Guide

Procedure

1. Include the **ICU** layer using the Workbench configuration editor.



2. Build the VSB project.

   During the build, the ICU source is downloaded (or accessed from the *installDir* **/download** directory) and placed in the VSB directory:

   vsbDir **/3pp/ICU/**

ICU is downloaded from the internet if necessary. The build will unpack ICU and apply patches to add VxWorks Support. The VSB build invokes the ICU supplied build system, wrapping it in the cross-build environment. The makefile that actually builds the code is **vxworks.lib.mak**. This is copied to the ICU build directory, *vsbDir* **/3pp/ICU/**. The ICU build system **install** stage will copy the headers and libraries to the standard locations in the VSB.

7. Testing Boost

Boost provides a set of regression tests that you can run to verify the functionality of its libraries on VxWorks.

Wind River runs the regression tests to validate Boost with VxWorks. The libraries that have successfully passed all tests are documented in the release notes. The tests can be built (and optionally run) as part of the VSB build, or the tests can be built and run separately.

To enable building the tests as part of the VSB build, enable the **BOOST_TESTS** component and any specific regression test components that you wish. All regression tests are dependent on the test harness library; therefore, it must be included first.

For example, to build math library tests on the command line:

```
$ vxprj vsb config -s -add "_WRS_CONFIG_BOOST_TESTS=y"
$ vxprj vsb config -s -add "_WRS_CONFIG_BOOST_MATH_TESTS=y"
```

The default behavior is to simply build the tests during the final VSB build stage. To actually execute the tests after each one is built, specify the telnet address of your test system.

```
$ vxprj vsb config -s -add "BOOST_TELNET_ADDR=192.168.200.1"
```

The target's path to the workspace defaults to using the TCF path, but you can set it to another method. The **_WRS_CONFIG_BOOST_HOST_FILEPATH_MAPPING_PREFIX** holds a prefix that the test harness will propagate to the test system; it should provide a path to the workspace as seen from the target's point of view. The prefix is prepended to the host path to create a path valid for the VxWorks target access of the test executables.

```
$ vxprj vsb config -s  \
-add "_WRS_CONFIG_BOOST_HOST_FILEPATH_MAPPING_PREFIX=wruserfedora:"
```

The tests are executed on a Linux host by connecting to the target through telnet using an expect script found in the Boost layer: **vxworks_boost_test_run.exp**. For Windows machines: **boost_test.py**.

You must create a VIP from the VSB before running the Boost tests; therefore, with this method, it is necessary to build the VSB twice in order to run the tests.

For example:

```
$ wrtool prj vsb create -bsp vxsim_linux simVSB
$ cd simVSB
$ vxprj vsb add BOOST
$ vxprj vsb config -s -add "_WRS_CONFIG_BOOST_MATH=y"
$ make
$ cd ..
$ wrtool prj vip create -vsb simVSB vxsim_linux llvm simVIP
$ cd simVIP
$ vxprj component add INCLUDE_IPTELNETS
$ vxprj component add INCLUDE_PX_SCHED_DEF_POLICIES
$ vxprj component add INCLUDE_POSIX_PIPES
$ vxprj component add INCLUDE_POSIX_SHM
$ vxprj component add INCLUDE_IPDHCPC
```

```
$ vxprj parameter set DHCPC_REQ_OPTS '"3"'
$ vxprj build
$ sudo ../../wrenv.linux -p vxworks-7
> vxsimnetd -sv &
> exit
$ gnome-terminal -e "vxsim -netif simnet0=192.168.200.1 -f default/vxWorks"
$ cd ..

$ cd simVSB
$ vxprj vsb config -s -add "_WRS_CONFIG_BOOST_TESTS=y"
$ vxprj vsb config -s -add "_WRS_CONFIG_BOOST_MATH_TESTS=y"
$ vxprj vsb config -s -add "_WRS_CONFIG_BOOST_TELNET_ADDR=192.168.200.1"
$ vxprj vsb config -s -add  \
"_WRS_CONFIG_BOOST_HOST_FILEPATH_MAPPING_PREFIX=wruserfedora:"
$ make
```

Note: If you are running Windows, in the above commands, replace wrenv.linux with wrenv and **vxsim_linux** with **vxsim_windows**.

Test pass/failure results appear on the console; the results are also stored in the Boost build directory in *vsbDir* **/3pp/BOOST/boost_1_71_0/bin.v2/libs/***library_name* .

Testing with build_run_tests.sh

As an alternative to running the Boost test harness within the VxWorks VSB build, a bash shell script, **build_run_tests.sh**, is generated by the build that can be invoked independently once the Boost layer is built.

For example:

```
$ wrtool prj vsb create -bsp simVSB vxsim_linux
$ cd simVSB
$ vxprj vsb add BOOST
$ vxprj vsb config -s -add "_WRS_CONFIG_BOOST_MATH=y"
$ vxprj vsb config -s -add  \
"_WRS_CONFIG_BOOST_HOST_FILEPATH_MAPPING_PREFIX=wruserfedora:"
$ make
$ cd ..
$ wrtool prj vip create -vsb simVSB vxsim_linux llvm simVIP
$ cd simVIP
$ vxprj component add INCLUDE_IPTELNETS
$ vxprj component add INCLUDE_PX_SCHED_DEF_POLICIES
$ vxprj component add INCLUDE_POSIX_PIPES
$ vxprj component add INCLUDE_IPDHCPC
$ vxprj parameter set DHCPC_REQ_OPTS '"3"'
$ vxprj component add
$ vxprj build
$ sudo ../../wrenv.linux -p vxworks-7
> vxsimnetd -sv &
> exit
$ gnome-terminal -e "vxsim -netif simnet0=192.168.200.1 -f default/vxWorks"
$ cd ..

$ cd simVSB/3pp/BOOST/boost_1_71_0
$ export BOOST_TELNET_ADDR="192.168.200.1"
$ ./build_run_tests.sh  --limit-tests=math
```

Note: If you are using a Windows system, in the above commands, replace wrenv.linux with wrenv and **vxsim_linux** with **vxsim_windows**.

You can also choose to edit the **build_run_tests.sh** script to set the host and target-relative paths and environment variables used during testing. The **LAYER_SRC_PATH** and **VSB_LD_LIBRARY_PATH** environment variables must be paths appropriate for the target (accessing the host file system), while the rest of the paths mentioned in the script are host paths. If you manually edit **build_run_tests.sh**, you do not need to add the **BOOST_TESTS** component to your VSB, nor do you need to add any of the specific Boost VSB regression test components.

The **build_run_tests.sh** script is a wrapper of the Boost b2 command, and by default, it tests the entire set of Boost libraries, including those that are unsupported; therefore, the command must be invoked with options that modify the behavior for your testing requirements. For complete b2 documentation, consult the Boost documentation at http://www.boost.org/build/doc/html/bbv2/overview/invocation.html#bbv2.overview.invocation.options.

The following options are not in **build_run_tests.sh**, but may be useful in modifing the behavior of the test build.

Option	Behavior
-q	Quit after the first test failure
-d2	Debug level 2, print command invocations
-a	Rebuild/re-run all specified tests, even those that have already passed
testing.execute=off	Build tests only, do not execute
--abbreviate-paths	Munge the test directory names so they are shorter
--limit-tests="*library1*|*library2*"	Only build and execute a subset of library tests
variant=release	Build test executables without debug information
-j*Jobs*	Run tests in parallel using *Jobs* threads

Scheduling Considerations

When running the Boost tests on VxWorks, Wind River enables *time slicing* in the scheduler. This allows some tests designed for fair-share scheduling to pass. The use of time-slicing prevents tasks of equal priority from starving each other.

Timeslicing is not required for any code using Boost. Time slicing should only be enabled in a VxWorks system after careful consideration of the real time impact.

For more information on scheduling considerations please see the *VxWorks 7 Application Programmer's Guide:Task Scheduling*.

6. BZIP2

1. Bzip2 Support

Bzip2 is a compression library and is provided as a VxWorks layer.

About This Task

You can create a VSB project with bzip2 support and then create VIP and RTP projects based on the VSB project.

Before You Begin

If you are on a system without Internet connectivity, you must obtain bzip2 sources and put them in the following directory:

installDir **/download**

If you make this *installDir* directory read-only, you must set the **WIND_3PP_DOWNLOAD_PATH** environment variable to an alternate directory location to store any downloads. Place the third-party source files in the alternate directory.

Obtain the bzip2 sources from the following location:

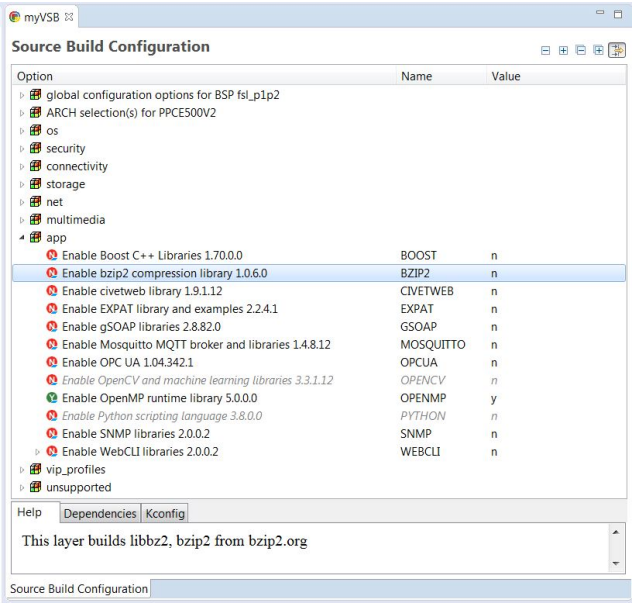
http://downloads.sourceforge.net/project/bzip2/bzip2-1.0.6.tar.gz

Check the source makefile in the **BZIP2** layer to verify the URL for your version of bzip2.

Procedure

1. Create a VSB project based on the appropriate VxWorks BSP and add the BZIP2 layer.

   You can use the Workbench VSB project Source Build Configuration view to enable the BZIP2 layer by setting the value for the bzip2 compression library to y.



2. Build the VSB project.

> **Note:** During the build, the source is downloaded from www.bzip.org and placed in the following VSB directory:
>
> *vsbDir* **/3pp/BZIP2/***bzip2-version*
>
> The source is patched for VxWorks, and the RTP and libraries are built and placed in the standard locations in the VSB directory tree.

3. Create and build a VIP project based on your VSB.

   There are no bzip2 specific components for the VIP.

7. OPENMP

1. OpenMP Support

VxWorks supports Open Multi-Processing (OpenMP) API specification for shared memory parallel programming in C and C++.

VxWorks supports OpenMP for the following CPU architectures:

- Intel (32-bit and 64-bit), using LLVM as a compiler
- ARM (32-bit and 64-bit), using LLVM as a compiler
- PowerPC (32-bit and 64-bit), using GCC as a compiler
- RISC-V (64-bit), using LLVM as a compiler

The C/C++ application is written as a single-threaded sequential program, which makes it easier to write, test, and debug. The decision to run the application as a single-threaded or multi-threaded application is made at the compile time. Add the compiler flag **-fopenmp** to all the files using OpenMP and where the system should allow the OpenMP runtime to use multiple OS-threads.

OpenMP-enabled applications do not require explicit thread management, as the OpenMP runtime will automatically decide how to distribute the application across multiple OS-threads.

Although it is possible, OpenMP applications should normally not make explicit calls to OS-specific routines for mutexes, semaphores, and other synchronization mechanism. Explicit synchronization of OpenMP managed threads is provided through the omp_lock_t type and through the master and critical OpenMP statements (see the OpenMP specification for details).

Most of the hardware used these days are multicore, making the runtime OpenMP applications more efficient.

VxWorks supports specific OpenMP features (see https://clang.llvm.org/docs/OpenMPSupport.html), except anything related to CUDA devices.

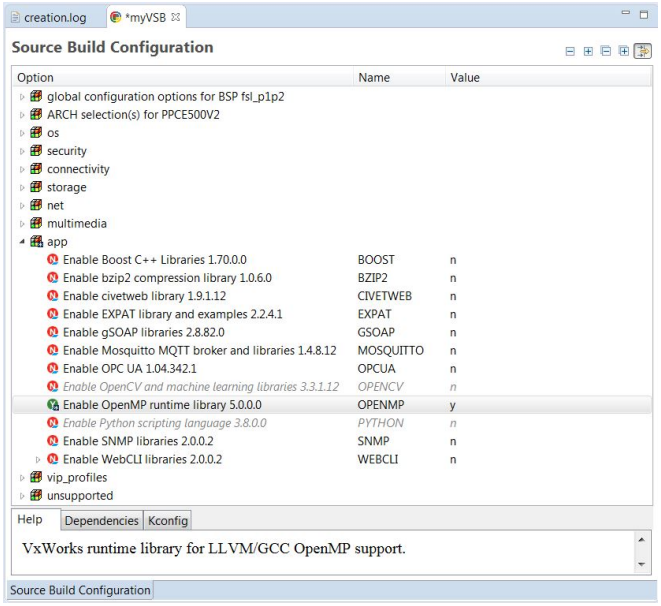
2. Configuring VxWorks with OpenMP Support

Create a VSB project with OpenMP support and create VIP and RTP projects based on the VSB project.

Procedure

1. Create a VSB project based on the appropriate VxWorks BSP and add the OPENMP layer.

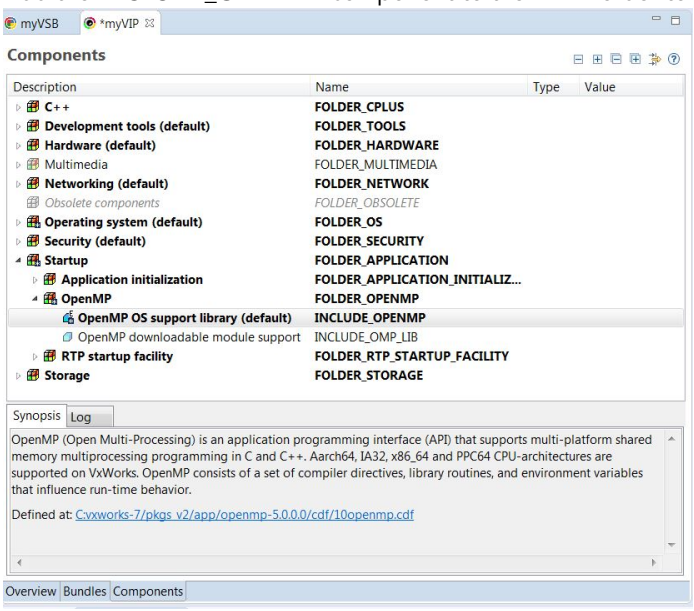
   You can use the Workbench VSB project Source Build Configuration view to enable the OPENMP layer by setting the value for the OpenMP runtime library to y.



2. Build the VSB project.

   Adding the OPENMP layer to the VSB, results in building the OpenMP runtime creating the library **libomp**.

3. Create a VIP project based on your VSB.
4. Add the **INCLUDE_OPENMP** component to the VIP in order to use OpenMP for VxWorks.



   VxWorks OpenMP is an opt-in feature on a per C/C++ file basis.

5. To support OpenMP applications packaged as DKMs, add the **INCLUDE_OMP_LIB** VIP component.

   This component is not required for OpenMP support in kernel or RTPs.

> **Note:** By default, OpenMP DKMs that include a variable definition for **threadprivate** will not load on 32-bit and 64-bit PPC architecture. To load these modules, you must include the -fno-common flag for compilation. See step 8 on page 28 for how to add tool flags to project properties.

6. Build the VIP.

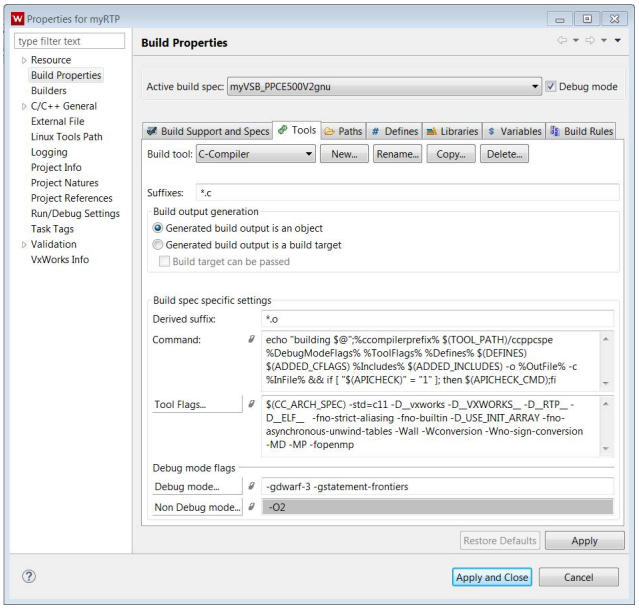
   OpenMP can be used both in kernel tasks and RTPs; however, for RTPs, OpenMP only supports static libraries.

7. Create an RTP and add the C/C++ application files to it.

   RTPs using OpenMP must explicitly link against the library libomp (i.e. add -lomp switch to the linker).

   By default, even with the **INCLUDE_OPENMP** component included, all the **#pragma omp** *<varname>* clauses are essentially no-operations. This makes it possible to build OpenMP enabled applications as single-threaded applications.

8. In order to generate a multi-threaded version of the application as instructed by the **#pragma omp** *<varname>* clause, add the compiler flag **-fopenmp** to the C/C++ files and build.
   - Right-click on the RTP project in the project explorer using Workbench and then click Properties.
   - Click the **Build Tools** tab on the Build Properties page, and set the **Build tool** to C/C++ -Compiler. Also set the **Active build spec** appropriately.
   - In the **Tool Flags** field, add the compiler option -fopenmp. Click on Apply and Apply and Close.
   - Build the RTP.



8. PYTHON

1. Python 3.8.0 Support

VxWorks supports **Python** 3.8.0 in the user space.

The following CPU architectures are supported:

- Intel (32-bit and 64-bit)
- ARM (32-bit and 64-bit) except for ARMv7 (see Limitations below)
- PowerPC (32-bit and 64-bit)

  Note: **Python** support for VxWorks 7 has not been validated for the PPC architecture as of SR0620.

- RISC-V (32-bit and 64-bit)

The implementation of **Python** for VxWorks is ported from **CPython**, the open-source reference implementation. Wind River has validated many **Python** modules and pushed the required changes to https://www.python.org/.

For additional Python functionality, these layers can be included in your VxWorks projects:

- Add the **SQLite** layer to use an embedded database in your **Python** application. For more information, see Adding SQLite to VxWorks with Python Support on page 32.
- Add the **PYTHON_NUMPY** layer to use scientific computing in your **Python** application. For more information, see Adding NumPy to VxWorks with Python Support on page 33
- The **LIBFFI** layer for **ctypes** is automatically added to your VSB if you enable the **PYTHON_NUMPY** layer. For more information, see Adding libffi for the ctypes Library to VxWorks with Python Support on page 34

Limitations

- **NumPy** does not work on the ARMv7 architecture because the ARMv7 libffi-3.2.1 code has pre-UAL instructions which is not supported by the LLVM compiler.
- These **Python** modules are not supported in VxWorks:
  - **crypt**
  - **curses**
  - **dbm**
  - **Graphical User Interfaces with Tk**
  - **grp**
  - **ossaudiodev**
  - **pty**
  - **readline**
  - **spwd**
  - **termios**
  - **turtl**
  - **asyncio**
  - **multiprocessing**

  - **netrc**
  - **lzma**
  - **venv**

- These methods in the VxWorks **os** layer are not supported:
  - os.popen( )
  - os.fork( )
  - os.execv( )
  - os.spawn*p*( )
  - os.forkpty( )
  - os.wait3( )
  - os.wait4( )
  - os.openpty( )

2. Configuring VxWorks with Python Support

Create a VxWorks Source Build (VSB) project with **Python** support and create a VxWorks Image Build (VIP) project based on the VSB project.

Before You Begin

If you are on a system without Internet connectivity, get the **Python** source and put them in the following directory:

installDir **/download**

If the *installDir* directory is read-only, you must set the **WIND_3PP_DOWNLOAD_PATH** environment variable to an alternate location to store any downloads. Place the third-party source files in the alternate directory.

Get the **Python** source from the following location:

https://www.python.org/ftp/python/3.8.0/Python-3.8.0.tgz

Check the source makefile in the **PYTHON** layer to verify the URL for your version of Python.

Procedure

1. Create a VSB project based on the appropriate VxWorks BSP using Workbench or from command line. For example:

```
$ cd InstallDir /workspace
```

```
$ vxprj vsb create -bsp <BSP> <VSB_DIR>
```

```
$ cd <VSB_DIR>
```

2. Add the **PYTHON** layer to the VSB.

```
$ vxprj vsb add PYTHON
```

3. Build the VSB project.

```
$ vxprj vsb build
```

The Python3 package for deployment is generated and put in: *<VSB_DIR>* **/usr/3pp/deploy**.

4. Create a VIP project based on your VSB. For example:

```
$ vxprj vip create <BSP> <TOOL> -vsb <VSB_DIR> <VIP_DIR>
```

```
$ cd <VIP_DIR>
```

5. Add the component **INCLUDE_PYTHON_SUPPORT** to the VIP in order to work with Python in the user space.

```
$ vxprj component add INCLUDE_PYTHON_SUPPORT
```

The component **INCLUDE_FILESYSTEM_SYMLINK_CONFIG** is automatically added to facilitate library searching and configuration settings when running Python on the target.

6. Set the **FILESYSTEM_SYMLINK_CONFIG_STR** parameter based on your deployment path to create symbolic links to Python executables. For example, if your deployment path is **/sd0a/deploy**:

```
$ vxprj component add INCLUDE_FILESYSTEM_SYMLINK_CONFIG
```

```
$ vxprj parameter setstring FILESYSTEM_SYMLINK_CONFIG_STR "<def>=/sd0a/deploy;/bin=<def>/bin;/u
sr=<def>/usr;/etc=<def>/etc;/lib=<def>/lib;"
```

Note: For more information on the deployment path, see Deploying and Testing Python on page 35.

7. (Optional) If you want to use Python command line arguments in the VxWorks interactive shell, add the **INCLUDE_SHELL** component.

```
$ vxprj component add INCLUDE_SHELL
```

8. (Optional) If your deployed media such as an SD card uses DosFs and you copied the Python contents from a Windows OS to the DosFs media, set the parameter **DOSFS_COMPAT_NT** to **TRUE** in your VIP. This ensures the Python interpreter starts up successfully.

```
$ vxprj parameter set DOSFS_COMPAT_NT TRUE
```

9. (Optional) If you want to enable user/password related modules (for example, getpass and pwd), configure VxWorks to support user management capabilities. For more information, see Configuring VxWorks to Support User/Password Modules on page 31.
10. Build the VIP.

3. Configuring VxWorks to Support User/Password Modules

If you want to enable user/password related modules (for example, getpass and pwd), you can configure VxWorks to support user management capabilities.

Adding a User and Login Capabilities

Procedure

1. Enter these commands to add the required components, set the hash key, hash key value and the user database file. Replace the example values with your own values for the UDB_HASH_KEY/UDB_HASH_KEY_LEN/UDB_STORAGE_PATH parameters.

```
vxprj component add INCLUDE_USER_IDENTIFICATION;
vxprj component remove INCLUDE_SHELL_SECURITY;
vxprj parameter set UDB_HASH_KEY "\"123456\"" ;
vxprj parameter set UDB_HASH_KEY_LEN 6;
vxprj parameter set UDB_STORAGE_PATH "\"/host.host/tmp/udb.txt\"";
```

2. Do the following to generate the udb.txt file and ensure it has been created.

```
-> udbReset()
-> userAdd("test","test",66,67)
-> cmd
[vxWorks *]# cat host.host/tmp/udb.txt
003 00000001 0042 Rf5GNbCLwzc7ZHhgsLbYFtjmuegZiFy7Px9wwnEPt3s=66 67 test IPeVAwAEAAA= QA6EGzzTp
FoEwTjUDy0N8WUUI804gM0iHx+HkkgzJCc= 1592450392 1592450293 0 0 0 1 0
```

   The udb.txt file is generated and is used for users to login into VxWorks. To use the secure shell, add the **INCLUDE_SHELL_SECURITY** component (removed in Step 1) to the VIP and rebuild it.

3. Use loginPrompt2 to log in as a specific user. If the shell is secured, VxWorks prompts you to log in before the shell opens.

```
-> loginPrompt2("test")
Password: value = 0 = 0x0
-> getuid
value = 66 = 0x42 = 'B'
-> getpwuid 66
value = 6450911152 = 0x1808117b0 = _func_loginPolicyVerify + 0x8
```

4. Start Python and execute the following in the Python shell.

```
>>> import pwd
>>> pwd.getpwuid(66)
pwd.struct_passwd(pw_name='test', pw_passwd='', pw_uid=66, pw_gid=67, pw_gecos='', pw_dir=None
, pw_shell=None)
```

   For more information about login capabilities, authentication, and managing users, see the *Vxworks 7 Security Programmers' Guide: User Authentication and Management*.

4. Adding SQLite to VxWorks with Python Support

Add the **SQLite** module if you want to use an embedded a database in your **Python** application. **SQLite** uses a nonstandard SQL query language to manage databases that are self-contained, transactional, and do not require a server or configuration.

Before You Begin

If you are on a system without Internet connectivity, get the **SQLite** source and put it in the following directory:

installDir **/download**

If the *installDir* directory is read-only, you must set the **WIND_3PP_DOWNLOAD_PATH** environment variable to an alternate directory location to store any downloads. Place the third-party source files in the alternate directory.

Get the **SQLite** source from the following location:

https://www.sqlite.org/2019/sqlite-autoconf-3300100.tar.gz

Create a VSB and VIP that includes **Python** support as described in Configuring VxWorks with Python Support on page 30.

Procedure

1. Add the **SQLITE** layer to the VSB.

```
$ vxprj vsb add SQLITE
```

2. Build the VSB project.

```
$ vxprj vsb build
```

3. Add the component **INCLUDE_SQLITE_SUPPORT** to the VIP so that **SQLite** works with **Python** in the user space.

```
$ vxprj component add INCLUDE_SQLITE_SUPPORT
```

4. Build the VIP.

5. Adding NumPy to VxWorks with Python Support

Add the **NumPy** package to use scientific computing in your **Python** application.

About This Task

Note: NumPy does not work on the ARMv7 architecture. For more information, see the Limitations section in Python 3.8.0 Support on page 29.

Before You Begin

If you are on a system without Internet connectivity, get the **NumPy** source and put it in the following directory:

installDir **/download**

If the *installDir* directory is read-only, you must set the **WIND_3PP_DOWNLOAD_PATH** environment variable to an alternate directory location to store any downloads. Place the third-party source files in the alternate directory.

Get the **NumPy** source from the following location:

https://github.com/numpy/numpy/releases/download/v1.18.1/numpy-1.18.1.tar.gz

Create a VSB and VIP that includes **Python** support as described in Configuring VxWorks with Python Support on page 30.

Procedure

1. Add the **PYTHON_NUMPY** layer to the VSB.

```
$ vxprj vsb add PYTHON_NUMPY
```

   The **PYTHON_NUMPY** layer is enabled and the **LIBFFI** layer is automatically enabled as well (the **libffi** library is required for **Numpy** to work).

2. Build the VSB project.

```
$ vxprj vsb build
```

3. Build the VIP.

6. Adding libffi for the ctypes Library to VxWorks with Python Support

You can add the **libffi** library to provide a bridge between your Python application and programs that are compiled into native code. **CPython** is the default implementation of the Python programming language and uses standard ctypes in the **libffi** library to call functions in DLLs or shared libraries.

About This Task

Note: If you enabled the **NumPy** layer, the **LIBFFI** layer is automatically enabled in your VSB. For more information, see Adding NumPy to VxWorks with Python Support on page 33.

Before You Begin

If you are on a system without Internet connectivity, get the **libffi** source and put it in the following directory:

installDir **/download**

If the *installDir* directory is read-only, you must set the **WIND_3PP_DOWNLOAD_PATH** environment variable to an alternate directory location to store any downloads. Place the third-party source files in the alternate directory.

Get the **libffi** source from the following location:

ftp://sourceware.org/pub/libffi/libffi-3.2.1.tar.gz

Create a VSB and VIP that includes **Python** support as described in Configuring VxWorks with Python Support on page 30.

Procedure

1. Add the **LIBFFI** layer to the VSB.

```
$ vxprj vsb add LIBFFI
```

2. Build the VSB project.

```
$ vxprj vsb build
```

3. Build the VIP.

7. Deploying and Testing Python

Copy the VxWorks generated Python source from the VSB directory to the deployment path on the target and run tests to ensure Python is working properly.

About This Task

The deployment path is where you put the Python package on the target side such as an SD card, USB disk, passFs, and so on. After you generate the Python package (see step 3 in Configuring VxWorks with Python Support on page 30), you deploy it by copying the package to a deployment path on a device (see step 1 of this procedure). For example, VxWorks generates the Python package and puts the output in the **<VSB_DIR>/usr/3pp/deploy** directory. If you copy the Python package **<VSB_DIR>/usr/3pp/deploy** to an SD card and VxWorks mounts the SD card as **/sd0a**, the deployment path is **/sd0a/deploy**.

The device deployment path must have sufficient space to contain the Python package. To improve performance, the device should be writable and have persistent storage.

Before You Begin

Configure Python as described in Configuring VxWorks with Python Support on page 30 including the optional step to add **INCLUDE_SHELL** to the VIP.

Procedure

1. Copy *<VSB_DIR>* **/usr/3pp/deploy** to your target device. Ensure the **FILESYSTEM_SYMLINK_CONFIG_STR** parameter matches the target path to create symbolic links to Python executables (see step 6 in Configuring VxWorks with Python Support on page 30).
2. Create a **helloworld.py** file on your host. In this example, the file is stored in the **/tmp** directory on the target file system.

```
# helloworld.py
import os
import sys
print("Hello World!")
```

> Note: If you launch Python from the shell, you may need to set the value of *rtpSpStackSize* before starting Python (for example, set the *rtpSpStackSize* to 0x1000000). For more information, see rtpSp( ) in the API reference.

3. Execute the **helloworld.py** file from the cmd shell once the target boots up:

```
-> cmd
```

```
[vxWorks *] cd /tmp
```

```
[vxWorks *] python3 helloworld.py
```

```
Hello World!
```

4. Launch Python from the cmd shell to print the `Hello World`! string:

```
[vxWorks *] python3
Launching process 'python3' ...
Process 'python3' (process Id = 0x805a9ad0) launched.
Python 3.8.0a4+ (default, Jun 122019, 10:59:47)
[Clang 8.0.0.1 (ssh://diabuild@stash.wrs.com:7999/llvm/clang.git 0a578b9ee67aa4 on vxworks
Type "help", "copyright", "credits" or "license"for more information.

>>> import os
>>> import sys
>>> print("Hello World!")
Hello World!
```

> **Note:** Your serial terminal must encode the backspace key as Control-H. If the backspace key is not mapped correctly (for example, it is encoded as Control-?), you must change the key mapping settings. The backspace code must be set properly to prevent READ, EVAL, PRINT, LOOP (REPL) errors.

5. Once the target boots, you can run various Python regression tests from the cmd shell.

   For example, you can run the regression test case **test_calendar.py** located in *<VSB_DIR>* **/usr/3pp/deploy/usr/lib/python3.8/test**:

```
[vxWorks *] python3 -m test -v test_calendar
```

```
Launching process 'python3' ...
Process 'python3' (process Id = 0x8127e120) launched.
== CPython 3.8.0a2+ (heads/master-dirty:2ff0fd0, Apr 25 2019, 15:18:45) [Clang 7.0.0.1 (ssh://d
iabuild@stash.wrs.com:7999/llvm/clang.git 41c373af7f0b90
== VxWorks_7-7.0.0-x86_Processor_-ACPI_BOOT_OP-_PAE-32bit little-endian
== cwd: /tmp/test_python_65536
== encodings: locale=UTF-8, FS=utf-8
Run tests sequentially
0:00:00 [1/1] test_calendar
test_days (test.test_calendar.CalendarTestCase) ... ok
test_enumerate_weekdays (test.test_calendar.CalendarTestCase) ... ok
test_illegal_weekday_reported (test.test_calendar.CalendarTestCase) ... ok
..................
test_formatweek (test.test_calendar.TestSubClassingCase) ... ok
test_formatweek_head (test.test_calendar.TestSubClassingCase) ... ok
test_timegm (test.test_calendar.TimegmTestCase) ... ok

----------------------------------------------------------------------

Ran 68 tests in 38.650s

OK (skipped=1)
test_calendar passed in 38 sec 834 ms

== Tests result: SUCCESS ==

1 test OK.
```

```
Total duration: 3
```

9. RUST

1. Rust 1.39 Support

VxWorks supports Rust 1.39 in user space.

Rust (https://www.rust-lang.org/) is a modern, productive, reliable, high-performance systems programming language which emphasizes safety and security. Rust provides a strong type system that makes it possible to detect memory errors and data races at compile time. VxWorks supports Rust with some limitations. For more information on usage and limitations please refer to Limitations and Additional Usage Information on page 41.

The following CPU architectures are supported:

- Intel (32-bit and 64-bit).
- ARM (32-bit and 64-bit).

> Note: Rust is only supported in configurations that enable hardware floating point (this is a default behavior in VxWorks for all of the above listed CPU architectures).

VxWorks Rust Targets

rustc compiler and **cargo** tool support the following Rust target triples, which can be selected by passing the --target option to **rustc** or **cargo**:

- ARM 32-bit: armv7-wrs-vxworks-eabihf
- ARM 64-bit: aarch64-wrs-vxworks
- Intel 32-bit: i686-wrs-vxworks
- Intel 64-bit: x86_64-wrs-vxworks

2. Configuring and Running Rust

A default VxWorks 7 installation includes a recent Rust toolchain based on a snapshot of Rust 1.39, together with pre-built Rust standard libraries for VxWorks.

About This Task

To use Rust in VxWorks you must create and build a VxWorks Source Build (VSB) project with Rust support, configure your VxWorks Image Project (VIP) and set some environment variables.

Procedure

1. Create a VSB project based on the appropriate VxWorks BSP using Workbench or from command line. For example:

```
$ cd installDir /workspace
```

```
$ vxprj vsb create -bsp BSP vsbDir
```

```
$ cd vsbDir
```

VSBs will automatically include the VxWorks layer **RUST** on targets that support Rust. This will also create an environment script rustenv that can be used to set up the environment for Rust development. However you do not have to use this script. For more information about additional configurations refer to Configuration for Advanced Users on page 40.

2. Build the VSB project.

```
$ vxprj vsb build
```

3. Create a VIP project including Rust support based on your VSB.

   Several VxWorks kernel components must be included in your VIP to run Rust applications. To include these components add the bundle **BUNDLE_RUST** to your VIP. For example:

```
$ vxprj vip create BSP TOOL -vsb vsbDir vipDir
```

```
$ cd vipDir
```

```
$ vxprj bundle add BUNDLE_RUST
```

4. Set up the environment for Rust development.

```
$ cd vsbDir/usr/rust
```

   If you are on a Windows host, run the command:

```
$ rustenv.bat
```

   If you are on a Linux host, run the command:

```
$ . rustenv.linux
```

5. Build a simple **hello world** Rust program.

```
$ cargo new hello
```

```
$ cd hello
```

```
$ cargo build
```

   This will create an RTP called **hello.vxe** in the directory **target/***vxworks-target***/***build-mode* where *vxworks-target* is one of the targets listed under section **VxWorks Rust Targets** in Rust 1.39 Support and *build-mode* is either **debug** or **release**. For example you might see an RTP in the directory: **target/armv7-wrs-vxworks-eabihf/debug/hello.vxe**

6. Run the **hello.vxe** on the VxWorks shell.

   You will see the following output:

```
Hello, world!
```

Note: For more information on using **cargo**, including how to customize the build output location or change the build mode from **debug** to **release**, see the cargo documentation here https://doc.rust-lang.org/cargo/guide/.

3. Configuration for Advanced Users

For more control over the build environment, you can set up the environment yourself instead of using the rustenv environment script.

More information on using the rustenv environment is described in Configuring and Running Rust on page 38.

Note: It is assumed that you have already set the standard VxWorks environment using wrenv.

VxWorks Compiler Driver (wr-c++)

The Rust toolchain uses a special VxWorks tool called **wr-c++** to find and link against the VxWorks libraries from your VSB. Ensure that the following environment variable is set:

[Windows]

```
set WIND_CC_SYSROOT=vsbDir
```

[Linux]

```
export WIND_CC_SYSROOT=vsbDir
```

PATH

By default the **PATH** set by wrenv includes the highest version Rust compiler that is installed. In the initial release of Rust for VxWorks there is only one Rust compiler. In the future you may have multiple Rust compiler versions installed. To select a compiler version other than the latest, add the location of that compiler to the **PATH** environment variable.

That location will be of the form:

```
WIND_HOME /compilers/rust-version /WIND_RUST_HOST_TYPE /bin
```

For example,

```
installDir /compilers/rust-1.39.0.0/x86_64-pc-windows-gnu/bin
```

Note: **WIND_HOME** and **WIND_RUST_HOST_TYPE** are standard variables set by wrenv. The standard VxWorks rustenv environment script always uses the Rust compiler version that is selected by your VSB.

CARGO_HOME

The cargo tool downloads any external crates that you use, to the directory specified by the environment variable **CARGO_HOME**. The standard VxWorks rustenv environment script sets **CARGO_HOME** to

```
vsbDir /usr/rust/.cargo
```

You can set **CARGO_HOME** to any directory you like, but it is recommended that you avoid using the same **CARGO_HOME** for your native desktop Rust installation (if any) and VxWorks Rust development.

$CARGO_HOME/config

(Optional) For convenience the standard pre-generated VxWorks Rust environment creates the configuration file under **$CARGO_HOME**

config:

```
[build]
target = "vxworks-target"
```

For example,

```
[build]
target = "armv7-wrs-vxworks-eabihf"
```

Note: This sets the default target used by cargo to the appropriate target for your VSB, avoiding the need to explicitly specify the --target flag on every cargo invocation

If you prefer to specify the target flag explicitly, you can invoke cargo as follows:

```
cargo build --target=vxworks-target
```

For example,

```
cargo build --target=armv7-wrs-vxworks-eabihf
```

4. Limitations and Additional Usage Information

VxWorks supports Rust with some limitations that you should be aware of.

Static and Dynamic Executables

VxWorks Rust executables are static by default, which means they only link against static libraries and do not support dynamic loading. Experimental support is provided for Rust dynamic executables. To build a Rust dynamic executable use the following combination of options with the Rust compiler **rustc**:

```
-C prefer-dynamic -C target-feature=-crt-static
```

Observe the - before crt-static in the second option. The first of these options selects Rust shared libraries, while the second selects VxWorks shared libraries. Just using one of these options (for example mixing Rust shared libraries with VxWorks static libraries) is not supported. When deploying a dynamic executable any required shared libraries must be available on the target. The Rust shared libraries can be found in the Rust toolchain installation under:

```
WIND_HOME /compilers/rust-version /WIND_RUST_HOST_TYPE /lib/rustlib/vxworks-target /lib
```

For example, the Rust shared libraries can be found under:

```
installDir /compilers/rust-1.39.0.0/x86_64-pc-windows-gnu/lib/rustlib/armv7-wrs-vxworks-eabihf/lib
```

Stack Size

When creating a new thread using std::thread or a new process using std::process::Command, the stack size of the newly created thread (which is a VxWorks pthread task under-the-hood) defaults to **256KB**. This value was chosen as a compromise between the much higher default Rust stack size in desktop environments (**2MB**) and the much lower default VxWorks pthread task size (**20KB**). To change this default stack size, set the environment variable **RUST_MIN_STACK**. You can also select a stack size when creating a thread using Builder. For more information see the Rust std::thread documentation (https://doc.rust-lang.org/std/thread/).

> **Note:** None of these settings affect the size of the initial task of a process created outside Rust (for example, from the VxWorks shell).

Standard Library Limitations

- The std::process library is incomplete and experimental.
- Out-of-stack detection is not yet implemented.
- Filesystem extensions write_at( ) and read_at( ) (https://doc.rust-lang.org/std/os/unix/fs/trait.FileExt.html) which are supported on most unix-like systems, are not yet supported on VxWorks. The APIs are provided but will always return an error.
- The following APIs require functionality that not available in VxWorks. For compatibility they are implemented as no-ops. Calls to these APIs will be silently ignored:
  - std::net::set_write_timeout
  - std::thread::set_name

cargo test and run

The cargo test and run commands need to run Rust executables, but since we don't currently provide a standard way to run a VxWorks RTP from a desktop environment, these commands do not yet work out-of-the-box for VxWorks targets. However you can specify your own custom runner to be used by cargo test or run by adding the following lines to your **$CARGO_HOME/config** file:

```
[target.'cfg(target_os = "vxworks")']
```

```
runner = "<some-program>"
```

This tells cargo to use the specified program to run executables when building for VxWorks.

Use of External crates

Rust is not just a language, it is an ecosystem. At the time of writing, almost 30,000 crates are available in the Rust package registry at https://crates.io/.

> **Note:** Because each crate depends on other crates and because some of those dependencies may be low-level crates that require operating system support, it is not possible to guarantee that any particular crate will work out-of-the-box with VxWorks. In many cases the changes required to support VxWorks will be small and over time Wind River will submit patches to add VxWorks support to crates that are found to be high-value by our customers.

Currently other than the core libraries included with VxWorks, the following crates include support for VxWorks:

libc `version >= 0.2.64`

At the time of writing, changes to support VxWorks are in the process of being incorporated into the following crate:

getrandom `(used by the rand crate)`

Calling Rust from C

Rust code (even if it appears to operate only on integers) may implicitly use floating point or vector instructions due to compiler optimizations. This requires any task that calls Rust code to be spawned with the **VX_FP_TASK** option set. All tasks in pure Rust applications automatically have this bit set. However if you want to call Rust from a C program, you must ensure that the calling task has **VX_FP_TASK** set.

10. VI EDITOR

A clone of the vi editor is available for use with VxWorks.

The ST Editor for VI Enthusiasts (STEVIE) version 3.7a.0.0 is provided in the VxWorks installation.

To configure VxWorks with the STEVIE vi editor, add the **UTILS_VI_EDITOR** layer to your VSB project, and **INCLUDE_VI_EDITOR** to your VIP project.

Use STEVIE from the VxWorks kernel shell (using the C interpreter) with the vi command. For example:

```
-> vi "/ram0/test.txt"
```

For command-line help, use:

```
-> vi_help
```

For more information about STEVIE, see http://nosuch.com/tjt/stevie/ and https://en.wikipedia.org/wiki/Stevie_(text_editor).

11. UNIX COMPATIBILITY LIBRARY

The UNIX compatibility library provides a shim layer for third party code.

This library is a collection of additional Unix functions typically used by open source libraries. The UNIX Compatibility library is provided to enable building specific versions of open source packages. Some third party projects mentioned in this guide use the UNIX compatibility library. Other recipe layers using the library are available on Wind River Github.

The UNIX compatibility library does not provide POSIX compliance, and there is no guarantee of specific behavior. It should not be used for new code.

The headers of the UNIX compatibility library are isolated from the normal public headers. A third party layer can use these additional headers by subscribing to the **UTILS_UNIX** layer in the **layer.vsbl** layer metadata file. For example:

```
Layer BOOST {
    SYNOPSIS          Boost C++ Libraries
    HELP              Boost provides free peer-reviewed portable C++ source libraries. \
                      This layer enables boost for VxWorks.
    VERSION           1.71.0.0
    FEATURE           OPEN_SOURCE
    VENDOR            Wind River
    VSB_REQUIRES      RTP
    LAYER_REQUIRES    UTILS_UNIX
    SUBSCRIBE      UTILS_UNIX
    PARENT
    OPTIONAL          YES
    DEFAULT           NO
    DOC_BUILD         NO
}
```

Note:

Code targeting 3rd party libraries may inherit a dependency to the UNIX compatibility library. Add the required definitions to your RTP or DKM project. Right-click on the application project name, then select **Properties > Build Properties > Includes** and add the required path, for example for RTPs:

```
-I$(VSB_DIR)/usr/h/published/UTILS_UNIX
```

For RTP projects switch to the **Library** tab and add the library also:

```
-lunix
```